

改进后的完整软件过程定义文档

1. 文档目的与适用范围

本文档面向“旧的怎么办”校园二手网项目，给出经过 AI 技术增强后的**完整软件过程定义**。文档在前期《软件过程定义》《AI技术应用场景匹配报告》《软件过程改进方案设计文档》《AI技术应用验证报告》的基础上，重新组织并正式定义本项目的软件过程体系，使其成为一份可直接用于课程作业提交和项目执行指导的统一文档。

本过程定义既继承了 ISO/IEC 12207 对软件生命周期过程的划分思想，也吸收了 ISO/IEC 15504、ISO/IEC 33001 对过程能力、过程结果与工作产品证据的关注方式。同时，文档将 2024 年以后 AI 软件工程（AI4SE）相关研究和行业报告中的成熟做法，剪裁后纳入本项目的实际执行流程中，形成一个适用于重点本科课程项目、具备现实可落地性的“人机协同增强型软件过程”。

本文档适用于以下活动：

- 校园二手网项目的需求分析、设计、开发、测试、发布与答辩准备；
- 项目团队 4 名成员在 6 个 Sprint 内的协同执行；
- 中期答辩、最终答辩、项目归档与过程说明材料编写；
- 团队在有限周期内采用 AI 工具提升研发效率与质量控制的全过程。

2. 过程定义来源

本次改进后的软件过程定义，主要来源于以下四类依据。

2.1 国际标准来源

第一，**ISO/IEC 12207**。该标准为软件生命周期过程提供了通用参考模型，覆盖项目过程、技术过程和支持过程。本项目过程划分中的项目策划与控制、需求获取与分析、架构与详细设计、实现与集成、测试与验证、发布与交付、评审与配置管理等，均来自这一标准框架的剪裁和映射。

第二，**ISO/IEC 15504 与 ISO/IEC 33001**。这两类标准强调过程定义不仅要写“做什么”，还要写清楚“为什么做”“做到什么程度算完成”“有哪些工作产品可以证明过程被执行过”。因此，本过程定义中每个关键过程都强调目的、输入、活动、输出、角色、门禁和度量指标，保证文档具有可评估性，而不是停留在原则描述层面。

2.2 项目实际来源

本过程定义的直接业务来源是已有的 [项目计划.txt](file:///c:/Users/Administrator/Desktop/软件过程与管理/作业/项目计划.txt) 和前期形成的 [软件过程.txt](file:///c:/Users/Administrator/Desktop/软件过程与管理/作业/软件过程.txt)。这些材料已经明确了：

- 项目目标：构建校园二手交易平台，覆盖 PC 网站与微信小程序；
- 团队规模：4 人；
- 组织方式：Scrum 敏捷迭代；
- 周期安排：6 个 Sprint；
- 里程碑：M0-M5；
- 技术架构：Spring Boot + Vue + 微信小程序 + MySQL + Redis；
- 核心功能：注册登录、商品发布与管理、搜索筛选、沟通、订单、评价、后台审核。

因此，本文档不是脱离项目实际的理论再设计，而是对项目已有过程的增强和重构。

2.3 AI 改进来源

本过程定义中的 AI 增强部分，主要来源于 [AI技术应用场景匹配报告.md](file:///c:/Users/Administrator/Desktop/软件过程与管理/作业/AI技术应用场景匹配报告.md) 和 [软件过程改进方案设计文档.md](file:///c:/Users/Administrator/Desktop/软件过程与管理/作业/软件过程改进方案设计文档.md)。前者用于识别 AI 在需求、设计、编码、测试、运维中的适配场景，后者用于明确如何把这些场景嵌入到原有 Scrum 过程之中。

2.4 真实验证来源

本过程定义不是纯理论构想，还吸收了 [AI技术应用验证报告.md](file:///c:/Users/Administrator/Desktop/软件过程与管理/作业/AI技术应用验证报告.md) 中的原型验证结论。该验证报告已使用真实 AI 工具，即当前使用的大语言模型助手（LLM），围绕“商品发布”功能进行测试用例生成对照实验，记录了提示词、执行过程、输出结果和人工对比数据，验证了 AI 在测试设计阶段的实际可用性。因此，本文档中的 AI 测试过程并非纸面想象，而是有原型验证支撑的改进项。

3. 剪裁与改进总原则

本项目是重点本科课程项目，团队小、周期短、目标明确，决定了过程设计必须遵循“够用、可执行、可交付、可解释”的原则。基于这一现实，过程剪裁与改进遵循以下原则。

3.1 轻量化原则

不照搬企业级全套制度，不引入大量审批节点和重型管理文档。所有过程活动都应尽量服务于当前项目交付目标，而不是为了“看起来规范”而增加额外负担。

3.2 交付导向原则

过程存在的核心价值是支撑项目按期交付，因此必须围绕 MVP 交付、中期答辩、正式版封版和最终验收来组织。不能出现“过程设计很完美，但项目做不完”的情况。

3.3 人机协同原则

AI 只作为增强工具，不取代项目经理、开发负责人、测试负责人和运维负责人。需求是否接受、设计是否采用、代码是否合并、缺陷是否关闭，都由人工做最终决定。AI 主要承担整理、补全、生成、校验、分析和提示等高重复活动。

3.4 高收益优先原则

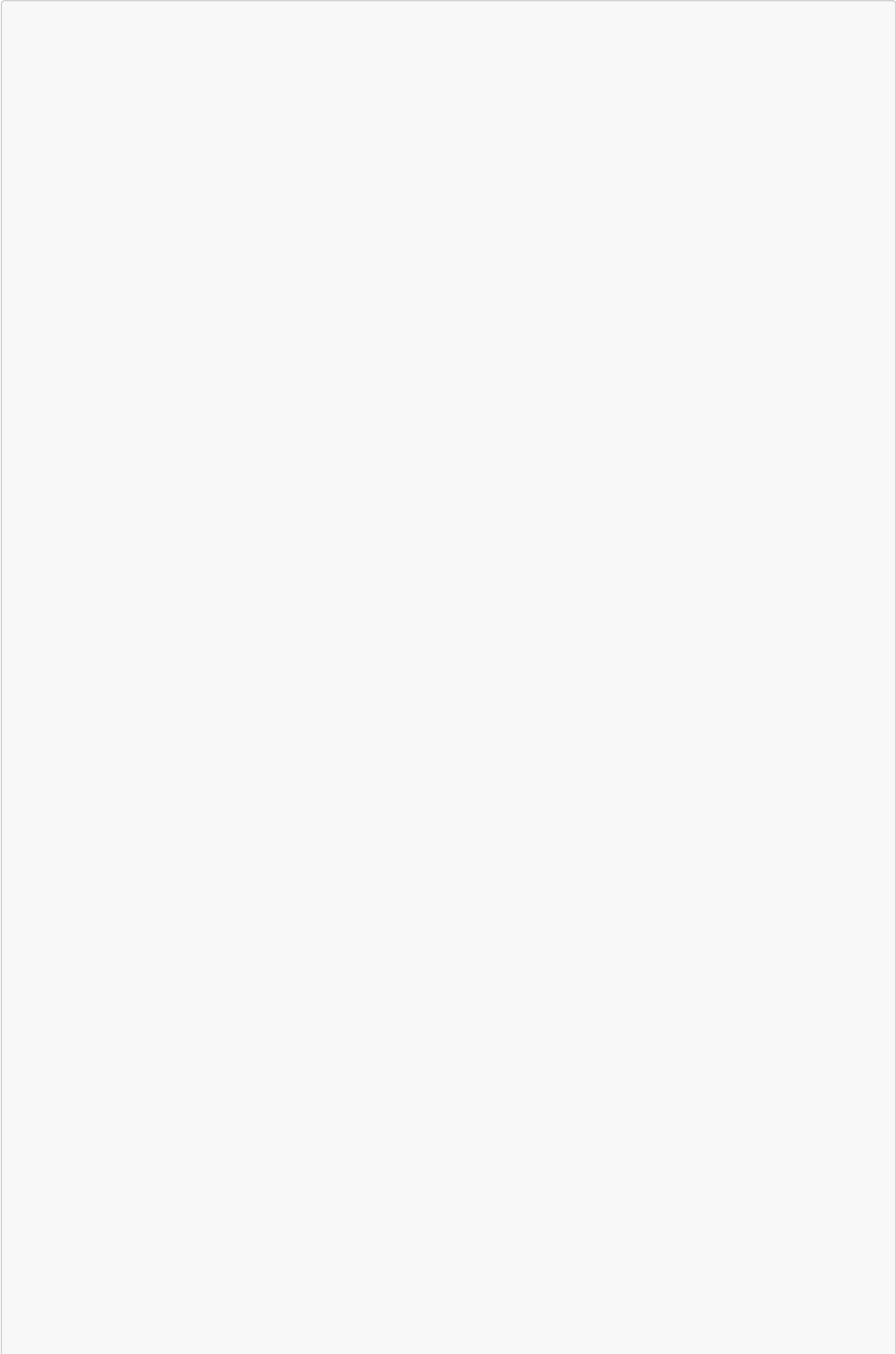
优先把 AI 用在最消耗时间、最容易遗漏、最影响质量的环节。基于前期研究与验证，本项目优先增强的环节依次为：实现与集成、测试与验证、需求分析、架构设计、发布前排障。

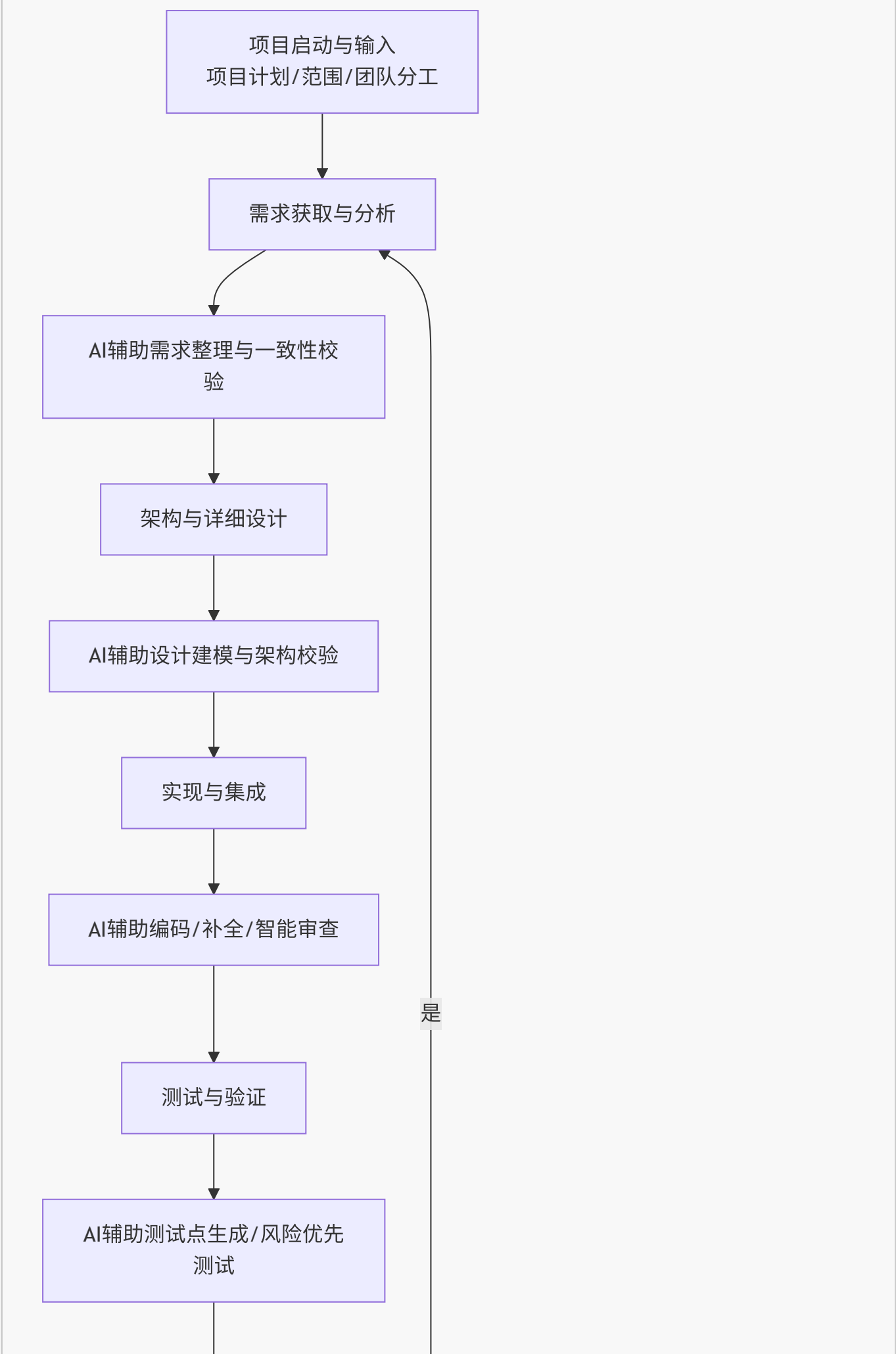
3.5 可复现与可度量原则

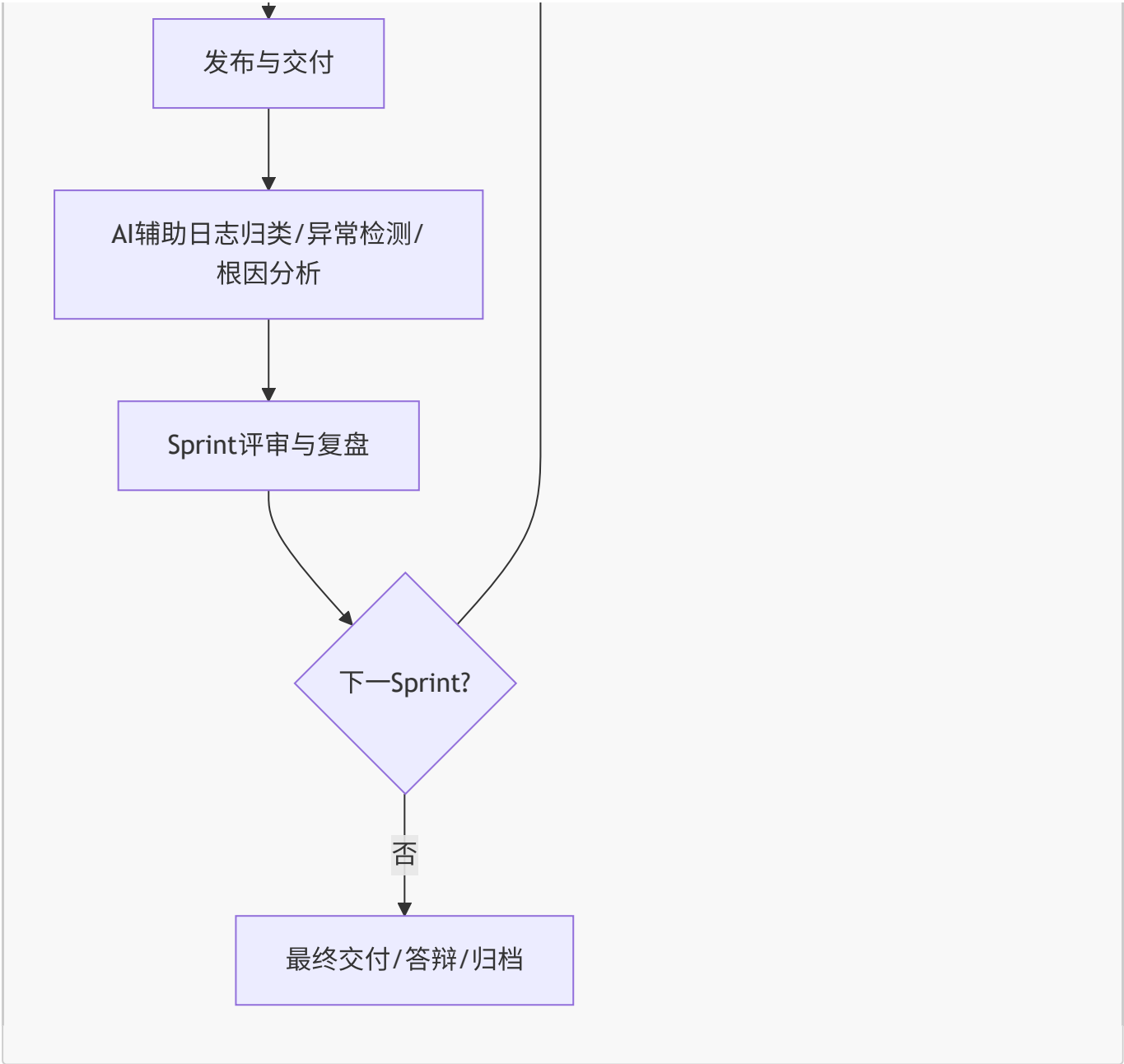
所有改进项都必须能说清楚“如何执行”，并且最好能给出可复现的步骤和效果指标。例如：如何用 AI 生成测试用例、如何做二轮提示补全、如何统计覆盖场景数和人工补充项数量等。

4. 改进后的总体生命周期模型

本项目仍采用 Scrum 敏捷开发模型，以 2 周为 1 个 Sprint，共 6 个 Sprint，但在原有生命周期主线中加入 AI 增强层与质量门禁层。总体模型如下：







在该模型中，真正变化的不是主流程顺序，而是每个阶段都有一个 AI 增强控制点，并在迭代末通过评审与复盘进行闭环调整。这样既保留了团队熟悉的工作方式，又提高了每个环节的质量前移能力。

5. 角色定义与职责

改进后的软件过程继续沿用原项目分工，并对 AI 使用责任做出明确规定。

5.1 项目经理 / 产品负责人

- 负责项目计划、里程碑控制、周报与答辩材料统筹；
- 负责需求整理、需求优先级排序和需求变更控制；
- 负责组织使用 AI 整理用户故事、验收标准和迭代目标；
- 负责在每个 Sprint 评审会上确认 AI 引入是否产生正向效果。

5.2 后端开发负责人

- 负责后端架构、接口规范和核心业务逻辑设计；
- 负责使用 AI 进行代码生成草稿、接口一致性检查和智能审查；

- 对 AI 输出的业务代码、数据库访问逻辑与安全风险承担最终复核责任。

5.3 数据 / 中间件 / 运维负责人

- 负责数据库结构设计、Redis 缓存方案、部署环境与性能优化；
- 负责使用 AI 检查数据字段一致性、日志异常模式和发布前风险点；
- 对缓存策略、性能问题和环境稳定性承担最终复核责任。

5.4 PC 前端负责人

- 负责 Vue 前端架构、页面实现与交互逻辑；
- 负责使用 AI 生成页面模板、补全表单校验逻辑、检查前后端接口字段映射；
- 对页面交互、组件设计和双端视觉一致性承担最终责任。

5.5 全体成员共同职责

- 不得直接照搬 AI 输出而不复核；
- 对关键业务术语、状态定义、测试结论和发布判断保留人工确认；
- 在 Sprint 复盘总结 AI 工具的有效场景和失效场景，持续优化使用方式。

6. 过程集合定义

改进后的完整软件过程分为三大类：项目过程、工程过程、支持过程。

6.1 项目过程

6.1.1 项目策划与控制

目的：建立项目边界、里程碑、人员分工和 Sprint 节奏，并对偏差进行跟踪纠正。

输入：项目计划、课程要求、团队分工、答辩时间节点。

活动：

- 形成整体项目计划和 6 个 Sprint 的阶段目标；
- 在每轮 Sprint 开始前确认范围和任务分配；
- 每周进行进度跟踪与问题记录；
- 对高风险任务进行提前识别与调整。

AI 增强点：

- 用 AI 生成 Sprint 计划摘要；
- 用 AI 整理会议纪要和行动项；
- 用 AI 归纳上轮问题与本轮风险。

输出：项目开发计划、Sprint 计划、周报、行动项清单。

门禁：迭代目标明确、责任人明确、交付物明确。

度量：计划偏差率、任务延迟数、风险关闭率。

6.1.2 配置与版本管理

目的：保证代码、文档、测试结果和交付版本可追溯。

活动：

- 采用统一命名的迭代版本；
- 对 MVP、中期版、正式版和最终版建立基线；
- 记录重要变更、缺陷修复和版本说明。

AI 增强点：

- 用 AI 生成版本说明草稿；
- 用 AI 总结本轮改动点与影响范围。

输出：版本记录、发布清单、封版说明。

度量：版本可追溯率、遗漏变更数。

6.2 工程过程

6.2.1 需求获取与分析

目的：将校园二手交易业务需求转化为结构化、可设计、可测试的需求输入。

输入：用户需求、讨论纪要、竞品观察、项目范围。

活动：

- 提取核心业务对象：用户、商品、订单、评价、审核；
- 明确业务流程：注册、发布、浏览、搜索、交易、评价；
- 形成用户故事与验收标准；
- 建立关键状态词汇表，如“待审核”“已上架”“已下架”“已成交”等。

AI 增强点：

- 用 AI 将非结构化讨论整理为用户故事；
- 用 AI 检查歧义词、冲突规则和遗漏场景；
- 用 AI 补充边界情况与验收标准草稿。

输出：PRD、需求清单、术语表、验收标准。

门禁：需求无重大歧义，关键状态定义清晰，测试可据此设计。

度量：需求返工次数、需求争议条目数、PRD 首稿形成时间。

6.2.2 架构与详细设计

目的：把需求转换为可实施的系统结构、模块划分、数据模型与接口定义。

活动：

- 明确 Spring Boot + Vue + 小程序 + MySQL + Redis 的分层架构；
- 形成数据库实体和字段设计；
- 输出接口说明；
- 说明缓存、权限和状态流转规则。

AI 增强点：

- 用 AI 生成用例图、类图、时序图草稿；
- 用 AI 对接口字段与数据结构做一致性检查；
- 用 AI 对缓存使用时机、状态机完整性和模块耦合做风险提醒。

输出：架构说明、ER 图、接口文档、关键流程图。

门禁：前后端能据此并行开发，关键流程设计可解释。

度量：联调前接口问题数、设计与实现不一致数。

6.2.3 实现与集成

目的：按 Sprint 节奏逐步实现核心功能并完成双端联调。

活动：

- 实现后端接口、前端页面、小程序页面；
- 完成商品、订单、评价、审核等模块集成；
- 联调接口和状态流。

AI 增强点：

- 用 AI 生成标准化代码草稿，如 CRUD、DTO、VO、表单校验；
- 用 AI 做提交前审查，检查异常处理、命名统一、字段映射、安全问题；
- 用 AI 识别重复逻辑和潜在重构机会。

输出：代码仓库、集成版本、构建结果。

门禁：主流程可运行，关键接口可联调，AI 生成代码经人工审查后合入。

度量：功能开发耗时、提交前发现问题数、联调阻塞次数。

6.2.4 测试与验证

目的：验证系统在功能、稳定性、安全性和异常处理方面满足需求。

活动：

- 进行单模块测试、联调测试、回归测试和全量测试；
- 验证正常流、异常流、边界值、状态流转和兼容性；
- 记录缺陷并闭环修复。

AI 增强点：

- 用 AI 根据需求与接口说明生成测试点初稿；
- 用两轮提示补充边界值与异常流；
- 用 AI 标记高风险模块，辅助排序测试优先级；
- 用 AI 归类失败结果和错误信息。

真实验证依据：

本项目已通过 [AI 技术应用验证报告.md](file:///c:/Users/Administrator/Desktop/软件过程与管理/作业/AI 技术应用验证报告.md) 验证“AI 生成测试用例”的可行性。验证中采用真实 LLM 工具，对“商品发布”功能进行两轮测试用例生成，对比人工方式与 AI 辅助方式，结果显示：

- 人工方式：9 条测试用例，6 个显式操作步骤；
- AI 辅助方式：16 条测试用例，2 轮提示；
- 单轮产出效率：AI 方式约为人工方式的 5.33 倍；
- 边界与异常场景覆盖明显优于人工初稿。

因此，该增强点已经具备可复现的原型验证基础。

输出：测试用例、测试报告、缺陷清单、回归记录。

门禁：核心路径通过，严重缺陷关闭或有明确处置意见。

度量：测试用例数、边界场景覆盖数、缺陷发现提前率、回归通过率。

6.2.5 发布与交付

目的：将系统发布到测试或生产环境，并形成答辩和验收可用版本。

活动：

- 环境搭建与部署；
- 发布前检查；
- 演示环境巡检；
- 交付材料准备。

AI 增强点：

- 用 AI 对日志、异常信息、接口报错做归类；
- 用 AI 进行发布前问题模式分析；
- 用 AI 辅助生成答辩版发布说明和演示风险提示。

输出：部署版本、用户手册、验收材料、演示脚本。

门禁：演示主流程可运行，核心功能稳定。

度量：发布前临时故障数、排障平均耗时、环境可用性。

6.3 支持过程

6.3.1 评审与复盘

目的：在每轮 Sprint 末确认交付物是否达标，并复盘问题和改进点。

活动：

- 进行 Sprint 评审，确认已完成内容；
- 梳理未完成项与原因；
- 总结 AI 工具在哪些任务上有效，哪些任务上不可靠。

输出：复盘记录、问题整改清单、下一轮改进建议。

度量：复盘问题关闭率、相同问题重复出现率。

6.3.2 文档与知识沉淀

目的：保证需求、设计、测试、部署等关键内容可复查。

活动：

- 更新 PRD、接口文档、测试记录和发布说明；
- 保留 AI 提示词模板和高价值使用样例；
- 保留可复现验证过程。

输出：文档集、模板集、验证记录。

度量：关键文档缺失数、验证材料可复现度。

7. 工作产品定义

改进后的完整过程要求形成以下工作产品：

- 项目开发计划；
- Sprint 计划与复盘记录；
- PRD 与验收标准；
- 术语表与状态定义表；
- 架构说明、ER 图、接口文档；
- 代码仓库与版本记录；
- 测试用例、测试报告、缺陷清单；
- 部署说明、发布说明、答辩材料；
- AI 使用记录：提示词样例、测试用例生成记录、AI 分析结论复核记录。

这些工作产品既支撑项目执行，也为老师检查“软件过程是否真实定义并被采用”提供证据。

8. 迭代落地方案

为保证可落地性，本项目不追求一次性把所有 AI 能力铺满，而采用分阶段落地。

8.1 第一阶段：需求整理与编码辅助优先

在项目前期，优先使用 AI 做需求整理、会议纪要归纳、验收标准补充和基础代码草稿生成，因为这些任务收益高、门槛低、易于被人工快速复核。

8.2 第二阶段：设计建模与测试点生成跟进

在项目中期，随着架构逐步稳定，引入 AI 建模辅助和测试点生成。此时接口、状态流和实体关系已经较清晰，AI 更容易生成高质量结果。

8.3 第三阶段：发布前分析和答辩巡检补强

在中期答辩和最终答辩前，重点使用 AI 对日志、测试结果和演示流程进行巡检，以降低临场风险。

9. 可复现验证说明

按照用户提出的技术要求，改进方案必须具备可落地性，原型验证必须使用真实 AI 工具，并给出可复现过程。本项目已满足这一要求。

9.1 使用的真实 AI 工具

本项目原型验证中实际使用的 AI 工具为：**当前使用的大语言模型助手（LLM）**。
该工具已被实际用于测试用例生成，不是想象中的未来工具，也不是未执行的方案草稿。

9.2 可复现步骤

复现“AI 生成测试用例”过程时，至少需要以下步骤：

1. 准备一个结构清晰的需求片段；
2. 将需求片段输入 LLM，并要求输出结构化测试用例；
3. 第二轮提示专门要求补充边界值、异常流、重复提交、上传限制等遗漏场景；
4. 将 AI 输出与人工初稿做对比，统计测试用例数、覆盖维度和人工补充项数量；
5. 由测试负责人进行人工复核后纳入正式测试集。

9.3 复现证据

相关提示词、输出摘要、对照数据和改进建议已经记录在 [AI技术应用验证报告.md] (file:///c:/Users/Administrator/Desktop/软件过程与管理/作业/AI技术应用验证报告.md) 中，因此该原型验证不是口头描述，而是可复查、可复用的过程证据。

10. 质量门禁与度量体系

为避免过程变成形式化存在，改进后的过程定义设置以下质量门禁：

- 需求门禁：关键业务状态定义明确，需求中无明显歧义，验收标准可用于测试设计；
- 设计门禁：接口文档与数据结构一致，关键流程图可支撑前后端并行开发；
- 开发门禁：主流程实现完成，AI 生成代码经过人工审查；
- 测试门禁：核心场景通过，边界/异常场景有覆盖，严重缺陷闭环；
- 发布门禁：演示环境稳定，答辩主流程通过巡检。

对应的核心度量指标包括：

- 需求返工次数；
- PRD 首稿形成时间；
- 联调前接口问题数；
- 单功能开发耗时；
- 提交前发现的问题数；
- 测试用例数与边界场景覆盖数；
- 严重缺陷发现提前率；
- 排障平均耗时；
- 答辩前临时故障数。

这些指标不要求像企业环境一样精确到平台级统计，但至少应在项目周报和复盘中进行简要记录，以证明过程改进确实带来效果。

11. 风险与控制策略

尽管 AI 能带来明显收益，但改进后的过程仍面临风险，必须明确控制策略。

11.1 AI 幻觉风险

AI 可能给出看似合理但实际上不符合项目上下文的需求、设计或测试建议。

控制策略：关键产物必须人工确认；AI 输出只作为候选结果。

11.2 过度依赖风险

团队可能把 AI 当作“自动完成器”，导致对业务逻辑理解不足。

控制策略：对关键需求、关键架构和关键缺陷仍要求人工解释与复核。

11.3 工具切换成本风险

如果过早引入复杂 AI 工具链，可能打断团队节奏。

控制策略：优先使用低门槛、高收益场景，逐步扩展，而不是一次性全面替换流程。

11.4 时间投入失衡风险

课程项目最重要的是完成高质量交付，而不是做“AI 工具秀”。

控制策略：所有 AI 活动都必须服务于当前 Sprint 交付，不能为了用 AI 而用 AI。

12. 结论

本文档给出的“改进后的完整软件过程定义”并不是对原有过程的推倒重来，而是在校园二手网项目既有 Scrum 框架上，建立一套更适合当前 AI4SE 发展趋势、又能在课程项目中切实落地的增强型软件过程。它的核心特点可以概括为三点：

第一，**结构完整**。文档覆盖了项目过程、工程过程和支持过程，并对每个关键过程定义了目的、活动、输入、输出、角色、工作产品、门禁与指标。

第二，**改进可落地**。AI 增强点不是抽象口号，而是明确嵌入到需求整理、设计建模、编码审查、测试点生成和发布前分析等现实活动中，符合 4 人团队和 6 个 Sprint 的执行节奏。

第三，**验证有依据**。至少一个核心改进场景，即“AI 生成测试用例”，已经通过真实 AI 工具完成最小可行验证，并形成可复现的验证材料。这使得本文档中的 AI 改进项具备实验支撑，而非停留在纯理论构想层面。

因此，本项目最终采用的软件过程定义应当以本文档为准：在继承 ISO/IEC 12207、15504、33001 思想的前提下，结合项目计划与 AI4SE 新实践，形成“Scrum 主流程 + AI 增强控制点 + 人工质量门禁 + 可复现验证闭环”的完整软件过程。对于重点本科课程项目而言，这一过程既足够规范，又不会脱离实际，能够同时支撑开发执行、作业说明和最终答辩展示。

13. 参考文献与行业资料

以下资料均为 2024 年及以后发表或发布，符合“至少引用 5 篇 2024 年以后的 AI 软件工程相关文献或行业报告”的要求。

- 靳东明, 金芝, 陈小红, 王春晖. [ChatModeler: 基于大语言模型的人机协作迭代式需求获取和建模方法](#). 《计算机研究与发展》, 2024.
- Gu, Siqi et al. [TestART: Improving LLM-based Unit Test via Co-evolution of Automated Generation and Repair Iteration](#). 2024.
- 中国信通院等. [《智能化软件工程技术和应用要求 第3部分：智能测试能力》](#). 2024.
- 中国信通院 AI4SE 工作组. [《AI4SE 行业现状调查报告（2024年度）》](#). 2025 发布.
- Beg, Arshad et al. [ACM Survey Draft on Formalising Software Requirements with Large Language Models](#). 2025.
- Giannouris, Polydoros et al. [NOMAD: A Multi-Agent LLM System for UML Class Diagram Generation from Natural Language Requirements](#). 2025.
- GitHub. [Review AI-generated code](#), 2025; [About GitHub Copilot code review](#), 2026; [GitHub Copilot tutorial](#), 2025.
- Testin 云测. [Testin XAgent: 五大能力开启测试新纪元](#). 2025.
- Gtest 全球软件测试技术峰会. [面向智能测试的接口平台构建](#). 2025.
- Forasoft. [AI in Software Architecture Design: Validate Scalability Early](#). 2026.